



UNIVERSITÀ DEGLI STUDI DI MESSINA
DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE, SCIENZE FISICHE E SCIENZE DELLA TERRA (MIFT)
MASTER IN DATA SCIENCE

Recommendation Systems Based on Public Datasets and Large Language Models

PTI La Rosa Francesco 22/09/2025 9.11.24

Author:	Parsa Kazemi (560 180)
Academic Supervisor:	Prof. Salvatore Distefano
Host Supervisor (FCRLAB):	Prof. Francesco La Rosa
Co-Supervisor:	Dr. Pierluigi Dell'Acqua

Academic Year 2025/2026

Total Internship Duration: 75 Hours

Abstract

The rapid expansion of user-generated content on digital platforms necessitates robust, real-time moderation systems. Traditional Static NLP classifiers provide the requisite processing speed but inherently lack contextual awareness, leading to high false-positive rates in domain-specific environments such as live-streams. This paper proposes a Modular Moderation Pipeline that addresses the trade-off between computational speed and contextual intelligence. The architecture decouples high-throughput data ingestion, powered by Apache Kafka and Spark, from high-intelligence reasoning. A lightweight DistilBERT model serves as the Tier-1 Static NLP Classifier, providing near-instantaneous inference (36.10 ms) for initial filtering. To resolve the contextual blindness of the static model, the pipeline introduces an independent Context Agent for dynamic metadata tagging and a Tier-2 LLM-based Agentic Auditor. Operating strictly on low-confidence predictions, the Agentic Auditor performs zero-shot forensic review to mitigate systemic bias and label noise. Experimental results on a

multi-genre dataset demonstrate that while the baseline Static NLP model generated significant label noise, the introduction of the Agentic Auditor significantly reduced false positives within the validation subset. The hybrid pipeline achieved an overall accuracy of 93.5%, showing strong agreement with human annotations. Furthermore, the modularity of the design suggests potential adaptability across diverse industries without structural re-engineering.

1 Introduction & Product Vision

The rapid expansion of digital platforms requires robust, automated systems capable of processing massive influxes of user-generated data in real-time. This project, conducted in collaboration with FCRLAB at the Università degli Studi di Messina, proposes a Modular Moderation Pipeline.

The overarching goal of this research is to explore hybrid methodologies that combine traditional machine learning techniques with the advanced reasoning capabilities of Large Language Models (LLMs). The primary objective is to construct an adaptable pipeline where predictive models and LLM

agents can be seamlessly swapped based on specific task requirements without re-engineering the underlying infrastructure.

1.1 Modular Pipeline Architecture

A core philosophy of this project is modularity. The system is designed to function as a plug-and-play product for advanced LLM-integrated recommendation systems. However, to rigorously stress-test the pipeline’s real-time ingestion capabilities, latency overheads, and contextual reasoning, this report evaluates a highly demanding proof-of-concept: Trust & Safety moderation (hate-speech detection in live-stream environments).

By evaluating the architecture against the strict latency and accuracy requirements of live moderation, the system’s robustness is proven. Because the architecture is modular, the exact same pipeline can be instantly repurposed back to its primary recommendation system objective, or even adapted for medical image segmentation, simply by swapping the primary prediction module and metadata parameters.

1.2 Limitations of Static Processors

To establish the baseline processor for the moderation use-case, a Static NLP Classifier (DistilBERT [4]) was fine-tuned using a comprehensive corpus of 52,972 labeled comments aggregated from established academic datasets (Davidson et al. [1], HateXplain [2], and ToxiGen [3]). However, static NLP models inherently suffer from a critical lack of environmental intelligence.

No matter how accurately a static model is trained, it remains blind to the context surrounding a statement. For example, in a gaming environment, the phrase “kill him” is often a benign instruction. In a political discussion, however, the same phrase constitutes a severe threat. Static models also struggle immensely with sarcasm, cultural nuances, and adversarial orthography (intentional misspellings).

By recognizing that context resides in the metadata rather than the text alone, this project introduces an independent Agentic Auditor to resolve these ambiguities, bridging the gap between high-speed static processing and high-intelligence contextual reasoning.

2 Related Work

This section positions the Modular Moderation Pipeline within the broader context of automated Trust & Safety research, highlighting the limitations of prior approaches and defining the novel architectural contributions of this system.

2.1 Static Transformer-Based Moderation Systems

Pure transformer architectures have become the industry standard for text classification. Models such as RoBERTa [7] and DeBERTa [8] offer robust token-level analysis through advanced pretraining and disentangled attention mechanisms.

Furthermore, domain-specific iterations like HateBERT [9], which are fine-tuned explicitly on abusive language, provide strong baselines for toxicity detection. However, a primary limitation of these systems is their static mathematical thresholds. While highly accurate on standardized datasets [1], they inherently lack the ability to dynamically scale moderation boundaries based on shifting environmental context (e.g., distinguishing gaming slang from political threats).

2.2 Context-Aware Moderation Approaches

To address contextual blindness, recent studies have focused on enriching the data itself. Datasets such as HateXplain [2] and ToxiGen [3] have advanced the field by providing explainable annotations and synthetic adversarial examples to help static models recognize implicit hate speech. While these approaches improve baseline accuracy, they typically require re-training the entire model whenever a new domain or metadata parameter is introduced, limiting their plug-and-play adaptability.

2.3 LLM-Assisted Moderation Pipelines

The introduction of Trust & Safety-specific Large Language Models, such as Meta’s Llama-Guard [10] and the broader LLaMA-3 family [11], has drastically improved contextual reasoning. These models can evaluate nuance far better than static transformers. However, utilizing high-parameter LLMs for synchronous batch processing creates immense computational bottlenecks, rendering them unsuitable for high-throughput live-stream environments without massive hardware investments.

2.4 Agentic AI Systems for Trust & Safety

Recent literature exploring Agentic AI, specifically the ReAct framework [12], allows models to utilize external tools to verify context before making a decision. While promising, most agentic safety systems operate entirely offline. Our architecture addresses this gap by isolating the computational burden of deterministic agents strictly to asynchronous, low-confidence edge cases, ensuring the primary ingestion queue remains unblocked.

3 Big Data Pipeline & Infrastructure

To ensure the Modular Moderation Pipeline can handle enterprise-level data streams and to guarantee scientific reproducibility, the entire underlying architecture was containerized using Docker and built upon industry-standard Big Data frameworks. This ensures high availability, fault tolerance, and horizontal scalability.

3.1 Producers

The first Tier of the pipeline relies on automated “Producers.” These are intelligent ingestion scripts capable of connecting to any available API or platform stream. Because the system is

Table 1: Comparison of Moderation Architectures

Approach	Model Type	Contextual Reasoning	Real-Time Capability	Explainability	Streaming Support
Standard Transformers (HateBERT, RoBERTa)	Static	Low	High	Low	Yes
Generative Safety Models (Llama-Guard)	LLM	High	Low	Medium	No
Offline Agentic Systems (ReAct)	Agentic	High	Low	High	No
Our Approach (Modular Pipeline)	Hybrid	High	High (Tier-1)	High (Tier-2)	Yes

modular, a Producer can be configured to scrape live YouTube chat, ingest continuous Twitter feeds, or securely download medical scan datasets. Once the Producer acquires the raw data, it is formatted into a strictly defined Universal JSON Schema payload and pushed into the processing stream.

3.2 Stream Processing: Kafka & Spark

To manage the high throughput of incoming data, the pipeline utilizes Apache Kafka (v7.5.0) [5], orchestrated by Apache Zookeeper (v3.9). Kafka acts as a distributed event streaming platform, queuing the incoming payloads from the Producers to prevent system bottlenecking.

The queued data is subsequently consumed by Apache Spark (v3.4.1) [6]. Spark serves as the distributed computing engine, feeding the text data into the designated Static NLP Classifier. By decoupling the raw data ingestion from the AI processing phase, the system guarantees that sudden spikes in data volume (such as a viral live-stream event) will not crash the predictive models.

3.3 Storage and Visualization: Elasticsearch & Kibana

As the Static NLP Classifier evaluates the data, the outputs (alongside all original metadata and latency tracking metrics) are written continuously to an Elasticsearch cluster (v8.10.2). Elasticsearch was chosen for its high-speed, full-text search capabilities, which are essential for the downstream Agentic Auditor and Assistant Agent that require rapid data retrieval. Finally, Kibana (v8.10.2) is attached directly to the Elasticsearch database, providing a live, visual dashboard of the data stream, model predictions, and system health metrics. Because the database updates in real-time, it acts as the centralized nervous system for all subsequent automated agents.

4 The Multi-Tiered AI Architecture

4.1 Tier-1: Static NLP Classifier

The foundational layer of the Trust & Safety pipeline is the Static NLP Classifier. This module is responsible for the initial, high-speed classification of all incoming data packets. To ensure a balance between computational efficiency and linguistic accuracy, a `distilbert-base-uncased` model was selected as the primary architecture.

4.1.1 Dataset Composition & Preprocessing (Dataset A)

To construct a highly robust training corpus, a heterogeneous dataset comprising exactly 52,972 labeled comments was aggregated. To satisfy requirements for both real-world diversity and adversarial robustness, the dataset was sourced from three academic repositories:

- **Davidson et al. (2017) [1]:** 24,783 samples, providing a high density of informal speech and platform-specific slang.
- **HateXplain [2]:** 19,229 samples. To ensure label hygiene, annotator disagreements were resolved via majority voting, and ties were dropped to reduce noise.
- **ToxiGen [3]:** 8,960 samples. This machine-generated dataset provided synthetic adversarial labels crucial for training the model against implicit hate speech.

The final dataset exhibited an intentional class imbalance reflective of real-world environments. The complete structural and preprocessing parameters of the training corpus are detailed in Table 2.

Table 2: Dataset A Composition & Preprocessing Parameters of DistilBert model

Parameter	Specification
Class Distribution	Offensive: 49.6% ($n = 26,274$) Normal: 31.5% ($n = 16,686$) Hate: 18.9% ($n = 10,012$)
Train / Val Split	80% / 20%
Random State	42
Max Token Length	128 (Optimized for 95th percentile: 234 chars)
Text Preprocessing	Forced truncation and padding

4.1.2 Classical Baseline Justification

Before committing to deep learning architectures, a classical machine learning baseline was established using TF-IDF vectorization (maximum 5,000 features). A Logistic Regression model (configured with `class_weight='balanced'` and `max_iter=20000`) achieved an overall accuracy of 76%. Notably, the classical model struggled significantly with the "Hate" class, yielding an F1-score of only 0.66. This baseline scientifically justified the necessity of upgrading to a transformer-based architecture.

4.2 Training Hyperparameters

The `distilbert-base-uncased` model was fine-tuned on a 43,000-sample subset. Training was executed for 2 epochs using the AdamW optimizer with a learning rate of 5×10^{-5} , a weight decay of 0.01, and 100 warmup steps to stabilize early training. Input sequences were truncated and padded to a maximum length of 128 tokens. Due to hardware constraints (detailed in Section 7.1), the training batch size was restricted to 8, while the evaluation batch size was set to 16. The resulting model achieved a generalized validation accuracy of 79.44% (F1: 0.796).

4.2.1 The Latency Benchmark

The primary advantage of the Tier-1 processor is its near-instantaneous inference speed. During live-stream simulation testing, DistilBERT achieved an average inference time of 36.10 ms. This speed is critical for real-time moderation, as it allows the pipeline to process thousands of messages per second without introducing visible lag for the end-users. However, this speed comes at the cost of contextual nuance.

4.3 The Context Agent

To address the contextual blindness of the Static NLP Classifier, the pipeline introduces an independent Context Agent. This module acts as the bridge between raw metadata and the advanced reasoning agents in Tier 2.

4.3.1 Dynamic Environmental Labeling

Rather than treating every data packet as an isolated string, the Context Agent extracts metadata from the original source (e.g., Video Titles, Thread Subjects, or Device IDs). It assigns two critical variables to every record before it reaches the database:

1. `env_domain`: Defines the overarching category (Gaming, Medical, Politics, etc.).
2. `env_strictness`: A classification (low, medium, high) that dictates how the downstream LLM should interpret ambiguous language based on platform rules.

4.3.2 Prompt Architecture and Output Constraints

To execute this dynamic labeling, the Context Agent operates using a zero-shot classification prompt. The prompt is engineered to enforce strict adherence to the predefined taxonomy while guaranteeing programmatic compatibility through constrained output formatting.

The exact system instructions passed to the Large Language Model are structured as follows:

System Role: You are an automated metadata classifier for a Trust & Safety data ingestion pipeline. Analyze the following metadata scraped from the source stream.

Task Requirements:

- **Determine the Domain:** Identify the `env_domain` (e.g., Gaming, Politics, Medical, Music, Sports, News, General). Be specific (e.g., if the metadata references Dota 2, the domain is 'Gaming/Esports').
- **Determine the Strictness:** Assign the moderation `env_strictness` (low, medium, high).
 - *Low:* Gaming, esports, casual streams (slang and aggressive gaming terms are expected).
 - *High:* Politics, news, medical (strict professional conduct expected).
 - *Medium:* General chatting, music.

Output Constraints: You must output ONLY a valid JSON object. No explanations. No markdown formatting.

Example:

```
{ "env_domain":  
"Gaming/Esports", "env_strictness":  
"low" }
```

By explicitly forbidding conversational filler and markdown wrappers, the pipeline ensures the agent's output can be seamlessly parsed into standard JSON objects and injected into the Elasticsearch database without triggering system errors.

Because the system is designed as a modular product, the Context Agent suggests potential adaptability across various domains. In a Gaming domain, the strictness is automatically lowered to allow for competitive slang. Conversely, if the pipeline were redirected to a different task, such as Medical Segmentation, the Context Agent could hypothetically ingest scan metadata (e.g., device type, patient age) instead of chat logs. This modularity indicates the system could potentially function across different industries without extensive rewriting of the core codebase.

4.4 Tier-2 : The Agentic Auditor

The Tier-2 Agentic Auditor serves as the high-intelligence overlay for the pipeline. It operates asynchronously, auditing the Elasticsearch database to identify and resolve instances of label noise generated by the Static NLP Classifier.

To maintain system efficiency, the Agentic Auditor does not evaluate every message. Instead, it targets records where Tier-1 exhibited low confidence (specifically scores under 80%) or high potential for domain-specific error.

Utilizing the Ollama framework with a `gpt-oss:120b-cloud` model alias, the agent performs a zero-shot forensic review. To ensure deterministic behavior as a classifier rather than a creative text generator, inference is strictly executed with a `temperature=0.0`. The agent receives the raw text, the Tier-1 DistilBERT prediction, confidence score, and the `env_domain` metadata. The exact system prompt forces the LLM to act as an Explainable AI (XAI) filter:

System Role: You are an XAI CONTEXT JUDGE - TRUST & SAFETY ADVISOR. Evaluate the Tier-1 Static Model’s prediction.

Forensic Rules:

- **Sarcasm & In-Game Events:** Override static models if the text is standard gameplay rhetoric/trash talk.
- **Evasion & Dogwhistles:** Actively hunt for leetspeak, symbol replacement, or political dogwhistles.
- **Cultural Context:** Adjust severity based on regional/cultural slang vs. high-strictness environments.
- **Instigator/Troll Detection:** Catch False Negatives by identifying passive-aggressive or manipulative text that static models classify as 'Normal'.

Task & Output Constraints: Decide if Tier-1 was CORRECT, FALSE POSITIVE, or FALSE NEGATIVE. You must strictly output a JSON object: `{"decision": "...", "explanation": "..."}.`

Unlike the Static NLP model, which only provides a static categorical label, the Agentic Auditor generates a natural language reasoning string stored in the `agent_explanation` field. This provides full auditability, allowing human moderators to understand exactly why a decision was overturned. As demonstrated in the system benchmarks, this agentic layer acts as a promising contextual reasoning mechanism for ambiguous data points.

4.5 The Assistant Agent

While the Agentic Auditor operates autonomously in the background to resolve label noise, the pipeline includes a user-facing Assistant Agent. This module provides a natural language interface for human analysts to interact with the big data stream.

4.5.1 System Boundaries and Tool Registry

To ensure the model remains specialized for Trust & Safety tasks, the Assistant Agent is governed by a strict Tool Registry. This prevents the model from hallucinating or performing irrelevant tasks, limiting its actions to a specific set of operational boundaries. The Orchestrator is forced to output a rigid JSON schema to trigger a tool, bypassing standard conversational generation. The primary tools include:

- **Universal Search:** Allows the analyst to query the Elasticsearch database for specific chat keywords. It utilizes `"fuzziness": "AUTO"` to automatically correct human typos when searching for specific authors.

- **Statistics Tool:** Dynamically calculates volume, label distributions, and real-time average latencies across different domains.
- **Policy Explanation:** Provides the analyst with the logical reasoning behind specific moderation decisions.

4.5.2 Targeted Behavioral Review Tool (TBR)

A critical distinction in this architecture is the difference between the background Agentic Auditor and the Targeted Behavioral Review Tool (TBR). While the Tier-2 Auditor audits low-confidence records automatically, the TBR is a forensic tool intentionally triggered by a Human Expert Auditor. An analyst can use this tool to audit a specific user account. The TBR retrieves a user’s entire recent history and performs a behavioral verdict, checking the frequency of “Normal” behavior versus “Toxic” outbursts. Crucially, the TBR actively patches the Elasticsearch database live via an `es.update` command, establishing a direct human-in-the-loop override mechanism.

5 Experimental Setup

To evaluate the real-world performance of the Modular Moderation Pipeline, the system was deployed across five distinct live-stream environments, ensuring a comprehensive test of contextual adaptability.

5.1 Hardware Optimization and Software Environment

Despite the complexity of the agentic approaches, the system was highly optimized to run on constrained hardware to prove the viability of edge deployments. The complete hardware configurations and standardized software stack are detailed in Table 3.

Notably, the strict 4GB VRAM memory constraint of the local GPU directly necessitated the `train_batch_size=8` hyperparameter used in the DistilBERT training phase. The entire pipeline was containerized using Docker to ensure cross-platform reproducibility.

Table 3: Hardware and Software Environment Specifications

Component	Specification
Processor (CPU)	Intel Core i5 (8th Gen, 4 physical cores)
System Memory	12GB DDR4 RAM
Graphics (GPU)	NVIDIA GTX 1650 (4GB VRAM)
Deep Learning Framework	PyTorch (v2.10.0)
NLP/Transformer Library	Transformers (v5.1.0)
Classical ML Library	Scikit-Learn (v1.5.1)
Containerization	Docker

5.2 Data Collection For Validation (Dataset B)

Data was initially extracted from five diverse live-stream genres. Approximately 18,000 raw records were ingested via Kafka to test high-throughput stability. Because processing

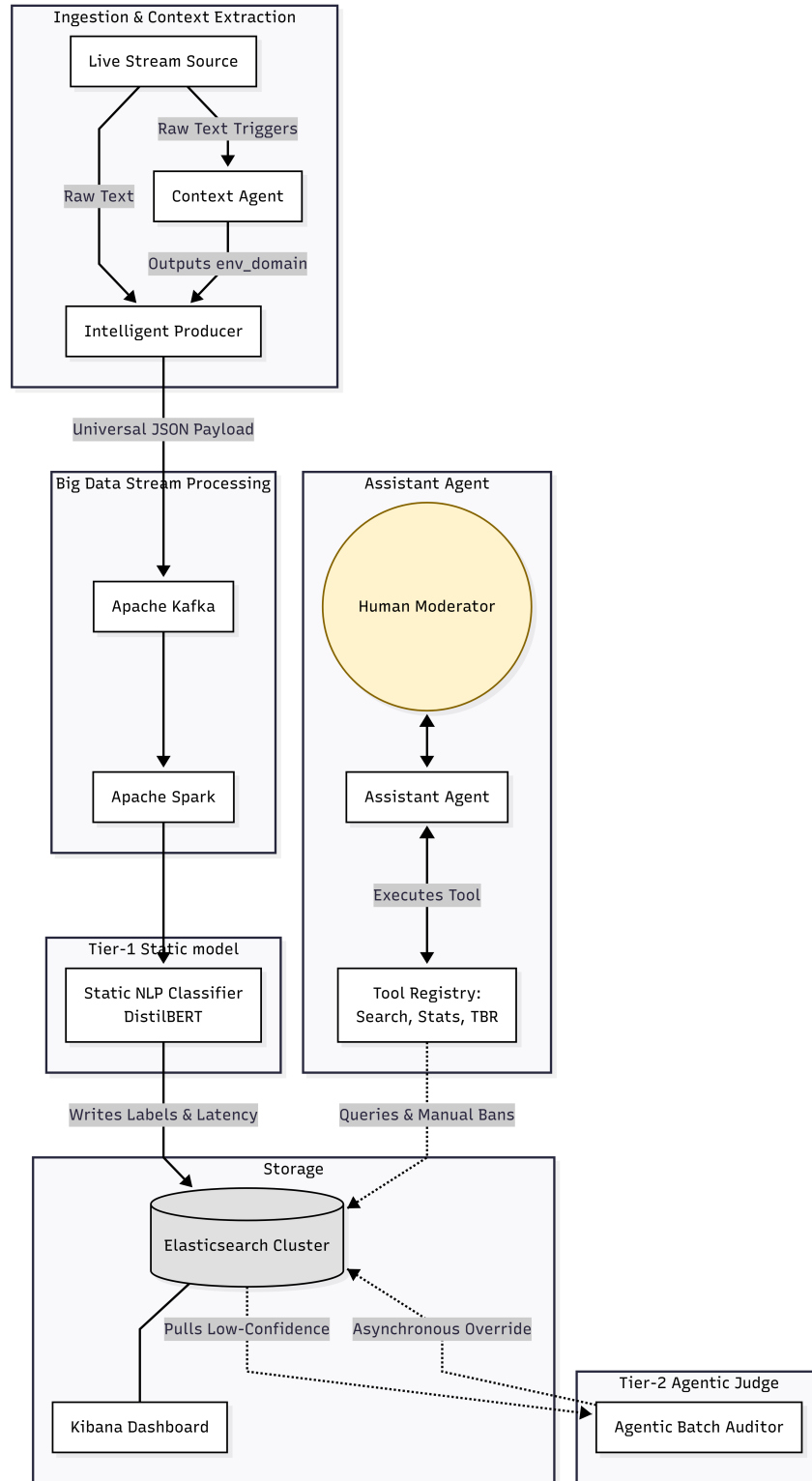


Figure 1: **The Modular Moderation Pipeline Architecture.** Data is ingested from live-stream sources and enriched with environmental metadata (Layer 1) before being queued and processed by Apache Kafka and Spark (Layer 2). The primary inference is executed by a high-speed Static NLP Classifier (Layer 3), with all latency and classification metrics written to an Elasticsearch cluster (Layer 4). Finally, low-confidence predictions are flagged for asynchronous review by the Tier-2 Agentic Auditor, while the Assistant Agent provides a natural language interface for Human Expert Auditors to execute final platform bans via the Tool Registry (Layer 5) for Trust & Safety enforcement.

18,000 records through a high-latency LLM is computationally prohibitive, a specialized validation subset of 459 low-confidence records (Dataset B) was extracted for the final benchmark.

This dataset exclusively targeted ambiguous messages where the Static NLP Classifier scored below 80% confidence. The complete source and class distribution of Dataset B, manually verified by a Human Expert, is detailed in Table 4.

Table 4: Class Distribution The Benchmark (n=459)

Label Class	Static Classifier	Agentic Auditor	Human Expert
Normal	347	384	384
Offensive	70	73	71
Hate	42	2	4
Total Errors vs. Human	133	30	Ground Truth

The evaluation dataset consists of 459 samples, primarily aggregated from digital streaming, gaming, and news environments. The following tables outline the source distribution of the text samples and the comparative class distributions across the manual (Ground Truth), synthetic baseline (Static Model), and synthetic refinement (Agent) layers.

Table 5: Dataset Composition by Source Domain

Environment / Domain	Sample Count	Percentage (%)
Gaming / Survival	340	74.1
Gaming (General)	51	11.1
News	30	6.5
Gaming / Esports	16	3.5
Politics / News	14	3.1
Music	8	1.7
Total	459	100.0

5.2.1 Multi-Genre Data Collection

Data was extracted from five diverse genres to maximize the variety of contextual challenges: Gaming, Music, Politics, Sport, and Education. Approximately 18,000 records were extracted via the Kafka ingestion pipeline. Each stream was sampled for a 20-minute duration to capture peak concurrent activity, demonstrating the pipeline’s capability to process high-throughput, unstructured data streams.

5.2.2 Label Distribution and Model Variances

Table 4 demonstrates the high false-positive rate of the Static Model, particularly in the “Hate” classification, when compared against the Human Ground Truth. To mitigate this label noise, Table 6 illustrates the intervention frequency of the LLM Agent when acting as a secondary contextual filter over the Static Model’s baseline predictions.

Table 6: LLM Agent Evaluation Outcomes

Agent Decision	Sample Count
Agreed with Static (Corrected)	312
Overturned Static (False Positive)	91
Flagged Static Miss (False Negative)	54
System / Parsing Error	2
Total Evaluated	459

5.3 The Comparison Methodology

Processing 18,000 records through a high-latency LLM is computationally prohibitive. Therefore, a specialized validation subset of 459 records was selected. This subset strictly targeted “low-confidence” messages, defined mathematically as any prediction where the Static NLP Classifier’s confidence score (C_{score}) fell below the threshold $\tau = 0.80$:

$$C_{score} < 0.80$$

This specific threshold was established to isolate the operational boundary where contextual label noise is most mathematically probable. To establish a scientifically valid benchmark for these edge cases, a three-way comparative methodology was employed:

1. **The Static Baseline:** Raw predictions generated by the fine-tuned DistilBERT model.
2. **The Agentic Overlay:** The reasoning and final verdicts provided by the LLM-based Agentic Auditor (configured via the `gpt-oss:120b-cloud` endpoint).
3. **Human Expert Ground Truth:** A domain expert manually reviewed the 459 records in isolation, assigning a baseline classification based on contextual analysis.

The human expert’s manual labeling served as the reference “Ground Truth.”

6 Results and Performance Analysis

By evaluating Dataset B of 459 manually labeled records against the human ground truth, a definitive comparison between static classification and agentic reasoning was established.

A primary objective of this research was to quantify the computational cost of contextual intelligence.

Table 7: Latency and Computational Overhead

Pipeline Layer	Raw Latency	Overhead
Tier-1 (Static)	36.10 ms	1.0x
Tier-2 (Agent)	6951.81 ms	192.6x

As detailed in Table 7, deploying the LLM introduces an Intelligence Overhead of 192.6x. While 6.95 seconds per inference is unacceptable for synchronous processing, utilizing the Agentic Auditor strictly for asynchronous low-confidence edge cases ensures the primary stream remains unblocked.

6.1 Accuracy and Label Noise Resolution

The baseline Static NLP Classifier, while high-performing in generalized contexts, exhibited severe limitations when faced with domain-specific slang. Out of the 459 validation records, the static model generated 326 correct predictions, yielding a baseline accuracy of 71.0%. The primary source of this degraded performance was label noise; the static model aggressively generated 83 false positives by incorrectly flagging innocent, domain-specific messages as toxic.

The application of the Agentic Auditor significantly mitigated this limitation. By applying environmental reasoning to the static model’s low-confidence predictions, the agentic overlay successfully identified and corrected the 83 false positives within this validation subset, significantly reducing the generated label noise. While the LLM occasionally exhibited leniency by generating False Negatives (allowing 30 subtly offensive messages to pass as Normal), its ability to completely halt unjust user bans raised the overall system accuracy to 93.5%.

6.2 Tripartite Agreement and Inter-Rater Overlap

To further understand the behavioral differences between static machine learning, agentic reasoning, and human cognition, a tripartite agreement analysis was conducted.

Table 8: Tripartite Agreement Rates

Evaluation Outcome	Agreement Rate
Human vs. Agent (Agent Accuracy)	93.5%
Human vs. Static (Static Accuracy)	71.0%
Static vs. Agent (AI Consensus)	68.4%
Unanimous Agreement (All 3 Match)	67.1%
Agent & Human Match (Static Failed)	26.4%

The 26.4% of records where the Agent and Human agreed, but the Static model failed, demonstrates that the LLM shows strong agreement with human annotations, helping to rescue innocent users when static token-weighting breaks down.

To illustrate the specific failure points of the static model, as well as the limitations and occasional hallucinations of the Agentic Auditor, Table 9 details five representative examples from Dataset B.

6.3 Failure Analysis of the Agentic Auditor

While the Agentic Auditor successfully corrected the static model’s false positives within the validation subset, the 30 introduced false negatives expose a critical limitation in the zero-shot prompting methodology: *implicit bias blindness*. A forensic review of the 30 failed samples indicates that the 120b parameter model struggled with semantic drift and heavily veiled sarcasm. In environments tagged with a `low` strictness domain (such as gaming), the agent exhibited an algorithmic over-correction, assuming that nearly all aggressive text

was benign trash talk. It failed to recognize highly specific, niche dogwhistles that require longitudinal historical context of a user’s behavior to decipher. This specific failure mode mathematically justifies the necessity of the manual Targeted Behavioral Review (TBR) tool; while the LLM is exceptional at clearing out high-volume label noise, detecting deeply embedded, implicit hate speech ultimately requires human temporal analysis.

7 Conclusion and Future Work

This research successfully demonstrates the viability of a Modular Moderation Pipeline for automated Trust & Safety environments. By decoupling high-speed data ingestion from high-intelligence reasoning, the system effectively mitigated the fundamental limitation of static NLP models: contextual blindness.

The three-way comparative benchmark demonstrated that while the DistilBERT [4] processor generated significant label noise (83 false positives), the introduction of an LLM-based Agentic Auditor significantly reduced this noise within the validation subset. The resulting 93.5% accuracy rate showed strong agreement with human annotations. Furthermore, the modular architecture suggests potential adaptability to different domains simply by swapping the primary Tier-1 processor.

7.1 Ethical Considerations & Dynamic Contextual Boundaries

Because this architecture executes automated moderation, addressing its ethical implications is a scientific necessity. First, the pipeline ensures a baseline of privacy by design; the Universal JSON Schema intentionally strips Personally Identifiable Information (PII) at the ingestion layer, relying strictly on User IDs, Messages, and Timeframes to limit the exposure of sensitive data to the LLM.

However, the definitions of safety, trustworthiness, and privacy are inherently context-dependent and cannot be treated uniformly across all modules. The current architecture evaluates safety based on platform moderation policies, which is sufficient for public live-streams where data is already publicly exposed. Yet, because the pipeline is designed to be domain-agnostic, repurposing it for high-stakes environments, such as medical image segmentation or financial auditing, drastically shifts the boundary of system safety.

In critical sectors, stripping PII is insufficient; the pipeline must enforce strict zero-retention policies to ensure sensitive inferences are not permanently cached in system memory or the Elasticsearch database. To achieve this, future implementations must evolve beyond static metadata tagging. The system requires **Dynamic Contextual Boundaries**. In this paradigm, the Tier-1 Context Agent would not merely identify the `env_domain`, but would actively inject domain-specific privacy constraints and memory-retention rules directly into the Tier-2 Agentic Auditor’s prompt. This ensures the LLM’s operational boundary (what it is allowed to read, process, and permanently store) flexes dynamically based on the severity of the ingestion source.

Table 9: Examples of Contextual Resolution and Agent Failures

Domain	Raw Text Message	Static Label	Agent Label	Ground Truth
Gaming	“awie would be better off working with the enemy”	HATE	NORMAL	NORMAL
Gaming	“Most adults are tired of that, viewbotz”	HATE	NORMAL	NORMAL
News	“Have you ever thought... ALL of you don’t have functioning BRAINS.. walai!”	HATE	NORMAL	OFFENSIVE
Politics	“the Jesus pic is of Trump Resurrecting Epstein”	NORMAL	NORMAL	OFFENSIVE
News	“Nigerians must remove the vampire government in 2027”	NORMAL	OFFENSIVE	NORMAL

Regarding algorithmic fairness, static NLP models inherently encode training biases, which frequently manifest as massive False Positive generation against distinct cultural dialects or gaming subcultures. The Agentic Auditor directly mitigates this systemic bias by applying equitable contextual reasoning to override the static model’s rigid mathematical thresholds. However, this introduces a trade-off: the LLM occasionally exhibits leniency, allowing false negatives (subtle toxicity) to bypass the filters.

To mitigate these risks, the architecture explicitly limits the Assistant Agent’s capabilities through a strict Tool Registry. Ultimately, the system mandates human oversight to resolve these architectural blind spots; automated components flag and explain anomalies, but only a Human Expert Auditor, utilizing the Targeted Behavioral Review (TBR) tool to evaluate the broader temporal context, possesses the authority to enact permanent platform bans.

7.2 Limitations and Future Work

The primary limitation of this architecture is the 192.6x latency overhead introduced by the LLM layer. While deploying the agent asynchronously mitigates pipeline bottlenecks, 6.95 seconds per inference remains computationally expensive. Furthermore, the agentic layer introduces unique vulnerabilities related to contextual misinterpretation. The LLM suffers from “screen blindness”; it understands the overarching domain but lacks access to the live audio/video feed, risking hallucinations if stream contexts abruptly shift.

To evolve this proof-of-concept into a comprehensive Master’s thesis, future work should consider the following architectural and experimental extensions:

Retrieval-Augmented Moderation (RAG): To move beyond zero-shot prompting, future extensions will explore Retrieval-Augmented Generation (RAG). Utilizing vector databases to retrieve historical moderation logs and semantic search over previous user behaviors will align the system with modern agentic AI standards.

Multi Agent Collaboration: The current architecture can be expanded into a collaborative multi-agent framework featuring specialized risk-scoring agents, behavioral profiling agents, and human-supervision escalation agents.

Edge Aware Optimization: To address the current computational overhead, future iterations will explore quantized

LLMs, lightweight edge-deployed models, batching optimization, and cost-aware inference orchestration.

Advanced Explainability & XAI: Future work will explore confidence-aware explanations, rationale extraction, and attention visualization to rigorously compare human and LLM reasoning paths.

Stronger Evaluation Pipelines: The validation methodology will be expanded to include extensive ablation studies, confusion matrices, adversarial evaluation, cross-domain generalization, and statistical significance tests.

Real-Time Memory & Temporal Context: Currently, the LLM evaluates messages mostly in isolation. Future architectures should incorporate short-term conversational memory, rolling context windows, and temporal user behavior tracking to improve the identification of complex dogwhistles and implicit toxicity.

Automated Model Distillation & Baseline Upgrades: The highly accurate outputs generated by the Agentic Auditor should be captured as a continuous “teacher” dataset to periodically fine-tune the Static NLP Classifier. Furthermore, future iterations should evaluate advanced baselines like RoBERTa [7], DeBERTa [8], HateBERT [9], or Llama-Guard [10] to determine if higher initial parameter counts reduce the downstream burden on the Agentic Auditor.

Federated Learning (FL): Finally, exploring decentralized FL mechanisms where lightweight predictive models are trained locally on edge devices and only aggregated model weights are transmitted back to a central server could vastly enhance user privacy while maintaining contextual intelligence.

References

- [1] T. Davidson, D. Warmesley, M. Macy, and I. Weber, “Automated Hate Speech Detection and the Problem of Offensive Language,” *Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM)*, 2017.
- [2] B. Mathew, P. Saha, S. M. Yimam, C. Biemann, P. Goyal, and A. Mukherjee, “HateXplain: A Benchmark Dataset for Explainable Hate Speech Detection,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 17, pp. 14867-14875, 2021.

- [3] T. Hartvigsen, S. Gabriel, H. Palangi, M. Sap, D. Ray, and E. Kamar, “ToxiGen: A Large-Scale Machine-Generated Dataset for Adversarial and Implicit Hate Speech Detection,” *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [4] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [5] J. Kreps, N. Narkhede, and J. Rao, “Kafka: a Distributed Messaging System for Log Processing,” *Proceedings of the NetDB*, 2011.
- [6] M. Zaharia et al., “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56-65, 2016.
- [7] Y. Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [8] P. He, X. Liu, J. Gao, and W. Chen, “DeBERTa: Decoding-enhanced BERT with Disentangled Attention,” *International Conference on Learning Representations (ICLR)*, 2021.
- [9] T. Caselli, V. Basile, J. Mitrović, and M. Granitzer, “HateBERT: Retraining BERT for Abusive Language Detection in English,” *Proceedings of the 5th Workshop on Online Abuse and Harms (WOAH)*, pp. 17-25, 2021.
- [10] H. Inan et al., “Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations,” *arXiv preprint arXiv:2312.06674*, 2023.
- [11] AI@Meta, “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *International Conference on Learning Representations (ICLR)*, 2023.



Parsa Kazemi
Author